MODELING TECHNIQUES

Applied Project in Big Data
on Industrial Dataset

MODELING TECHNIQUES

Few hints

1 Prediction fine payments

Many instances time-series data

2 AutoML demo

AutoGluon framework for text and tabular data

https://github.com/vgarshin/applied_project_course_25

MANY INSTANCES TIME-SERIES DATA

Understanding the data /1

You may face that sort of data
- diploma projects
- everyday work
- research

…it can be stock data

timestamp	some_asset	price	some_parameters
01/01/2020	A1	5500	…
02/01/2020	A1	6000	…
…	A1	…	…
31/12/2023	A1	7000	…
01/01/2020	B2	200	…
…	B2	…	…
31/12/2023	B2	200	…
…	C3	…	…

…or users' data

timestamp	some_user	log
01/01/2020	User1	log01
…	User1	…
31/12/2023	User1	log02
01/01/2020	User2	log03
…	…	…
01/01/2020	User3	logXX
…	…	…

…and there is A LOT OF DATA (million of rows, hundreds of assets / users)

MANY INSTANCES TIME-SERIES DATA

Understanding the data /2

What can go wrong with that sort of data? Seems like regular time-series problem, but

Point 1 = Many instances to build time series data for

Point 2 = Time data can have gaps

Point 3 = Time series can be short

Point 4 = Time series model could be heavy

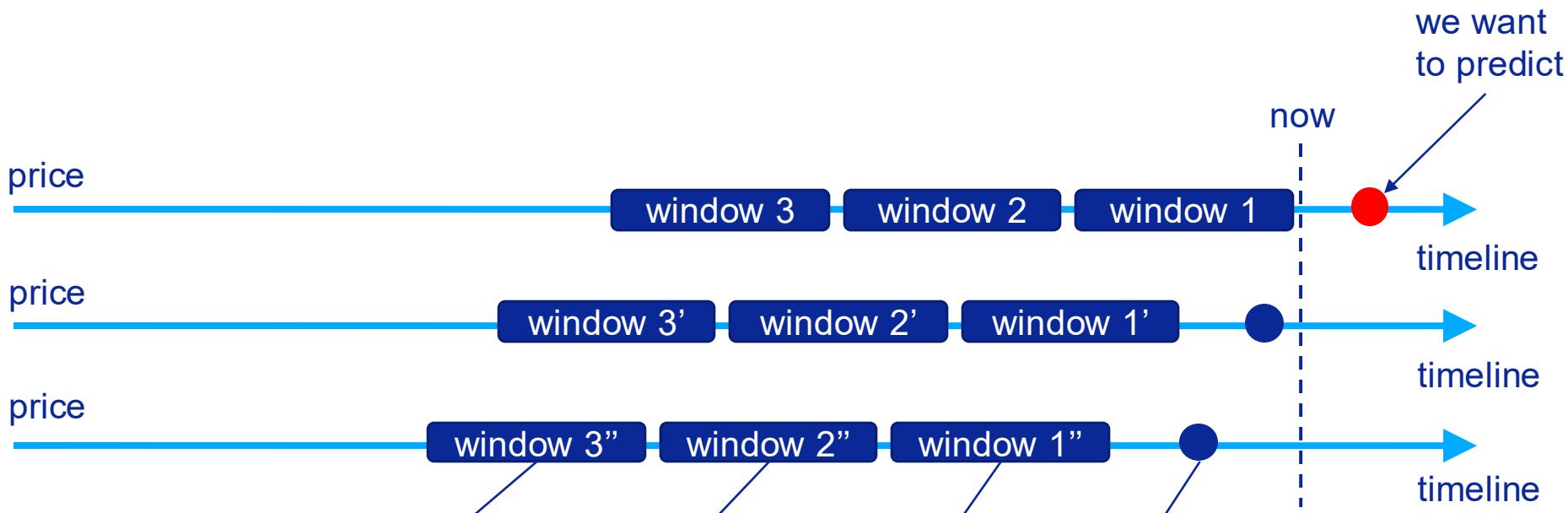
Point 5 = Instances can affect each other

MANY INSTANCES TIME-SERIES DATA

Idea

Let's create something that looks like time-series data but for decision-tree algorithms

e.g. we want to predict price for the assets



lag_feature_1	lag_feature_2	lag_feature_3	target
F(window1')	F(window2')	F(window3')	p1
F(window1'')	F(window2'')	F(window3'')	p2
…	…	…	…

You may use
- different windows
- different F() functions (mean, std, list of string)
- move windows to deep past

MANY INSTANCES TIME-SERIES DATA

PySpark code example

```
d1 = (Window()
      .partitionBy(F.col('base_symbol'))
      .orderBy(F.col('date').cast('timestamp').cast('long'))
      .rangeBetween(-days(1 + 1), -days(1)))

d2 = (Window()
      .partitionBy(F.col('base_symbol'))
      .orderBy(F.col('date').cast('timestamp').cast('long'))
      .rangeBetween(-days(2 + 1), -days(1)))

d3 = (Window()
      .partitionBy(F.col('base_symbol'))
      .orderBy(F.col('date').cast('timestamp').cast('long'))
      .rangeBetween(-days(3 + 1), -days(1)))
```

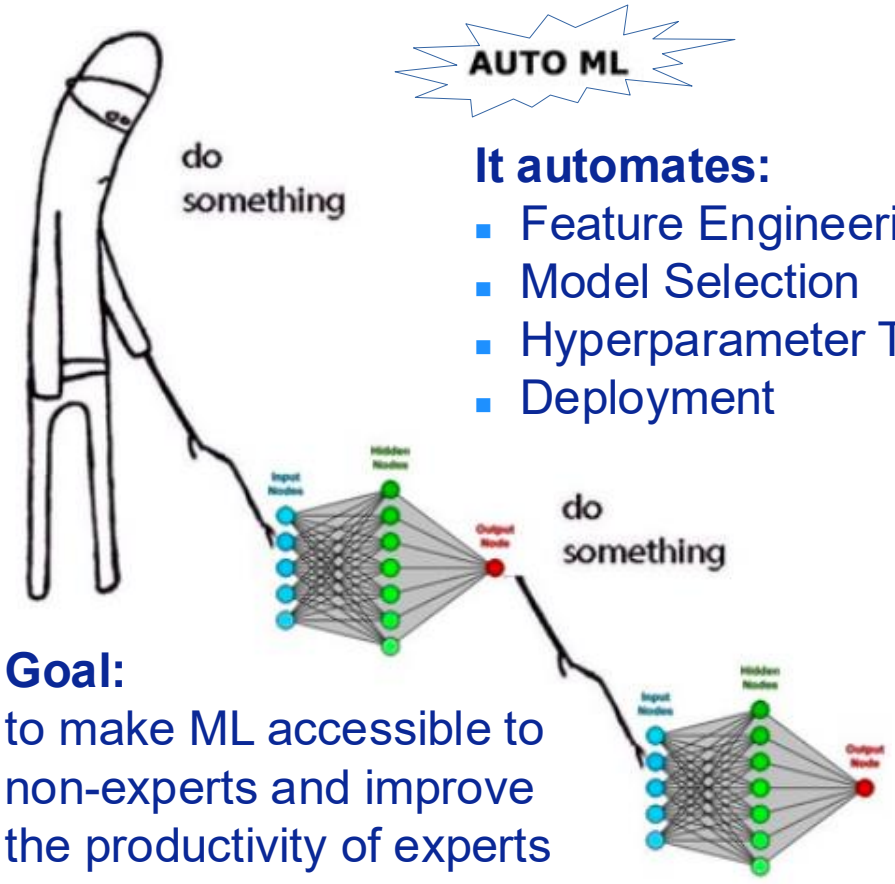
```
w1 = (Window()
      .partitionBy(F.col('base_symbol'))
      .orderBy(F.col('date').cast('timestamp').cast('long'))
      .rangeBetween(-days(7 + 1), -days(1)))

w2 = (Window()
      .partitionBy(F.col('base_symbol'))
      .orderBy(F.col('date').cast('timestamp').cast('long'))
      .rangeBetween(-days(2 * 7 + 1), -days(1)))
```

…and function:
F() = mean (), stdev()

EXPLAINING AUTOML

Automating the Machine Learning Process



It automates:

- Feature Engineering
- Model Selection
- Hyperparameter Tuning
- Deployment

Goal:
to make ML accessible to non-experts and improve the productivity of experts

Pros (Hype)

- ✅ democratizes AI
- ✅ huge time savings
- ✅ often better performance
- ✅ provides strong baselines
- ✅ standardizes workflows

Cons (Reality)

- ❌ can be a "black box"
- ❌ computationally expensive
- ❌ risk of overfitting & misuse
- ❌ limited customization
- ❌ doesn't replace expertise

Framework	Key Characteristic	Best For
AutoGluon	Ease of use, strong tabular data performance	Getting the best model with minimal code
Google AutoML	Cloud-based, user-friendly UI	Non-programmers or cloud-centric workflows
H2O AutoML	Scalable, Java-based, strong in enterprise	Large datasets and distributed computing

EXPLAINING AUTOML

How To

!pip install autogluon

https://auto.gluon.ai/

!pip uninstall h2o

https://docs.h2o.ai/h2o/latest-stable/h2o-docs/index.html

APPLIED PROJECT

Next seminar

You

…show us:

1) Framework you have chosen
2) Modeling pipeline (data preprocessing, feature generation, train-test-validation split strategy, model's train and validation)
3) Baseline model

We

…tell you about experiments with models:

1) How to store results
2) What tools can be used to track modeling process
3) Optimizing ML models
4) Automated reporting for drift detection and performance dashboards